DESF5 - Desafio Final

Cássio Esteves de Oliveira

Matricula: 0000178207

Turma: 2025-11A - POS - Engenheiro(a) de Dados

Sumário

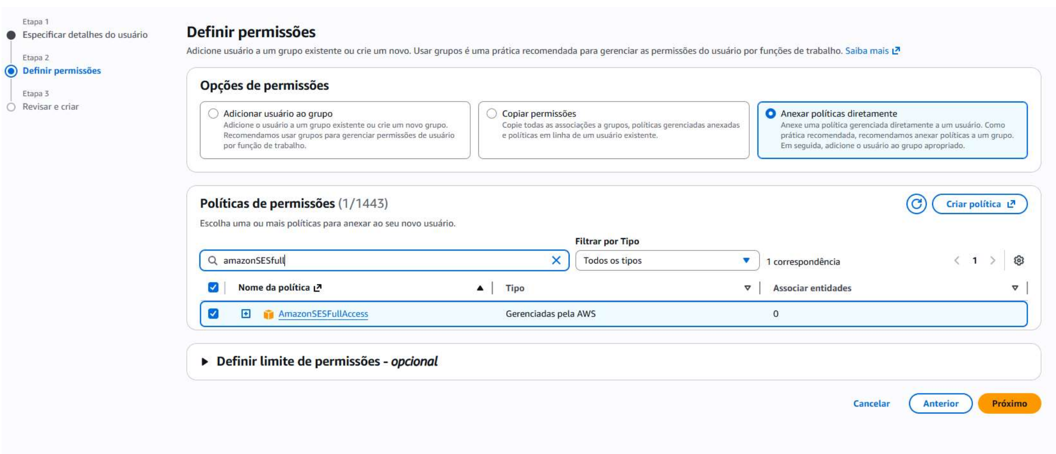
PASSO 1 — Criar um usuário IAM:

Console AWS → Pesquise por IAM → Users → Criar usuário → Dê nome para o usuário → "Próximo" → Determine as políticas → "Próximo" → Revise as informações → "Criar usuário".

Nome do usuário

kafka-spark-cesteves

Permissões: amazonSESfull



PASSO 2 — Criar as credenciais de acesso:

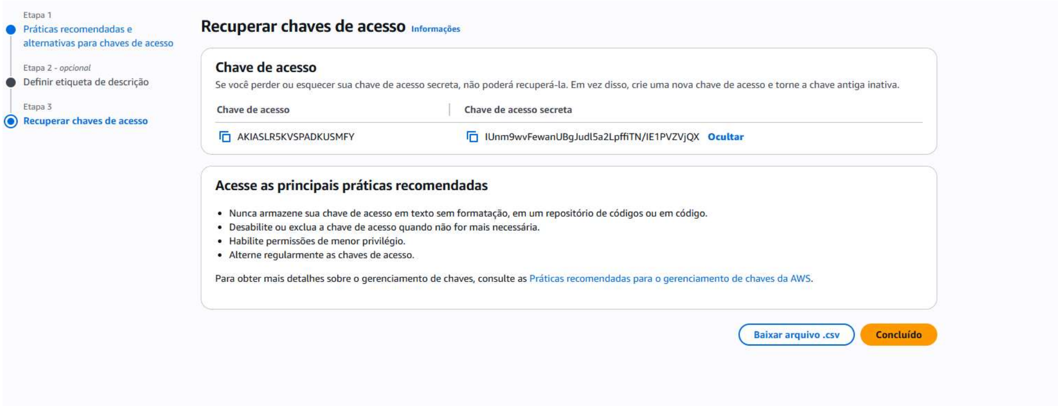
Acessar o usuário criado → Aba: Credenciais de segurança → "Criar chave de acesso" → Selecione "Aplicação Executada fora da AWS" → "Próximo" → "Criar chave de acesso" → Armazene os dados **Chave de acesso** e **Chave de acesso secreta** → Concluído.

Criar chave de acesso

Chave de acesso: **AKIASL....**

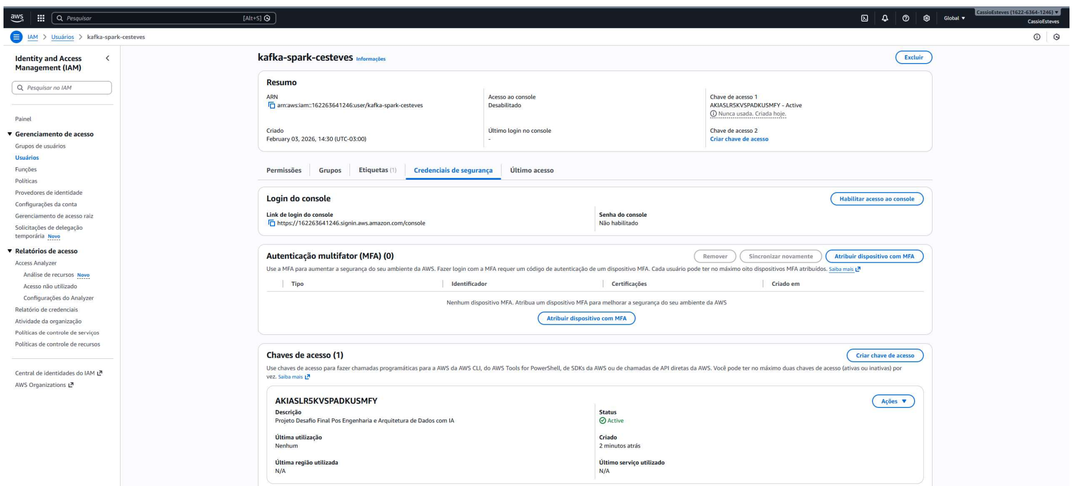
Chave de acesso secreta: **IUnm9...**

Chave salva em: L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD



Login do console

Link de login do console: https://162263641246.signin.aws.amazon.com/console

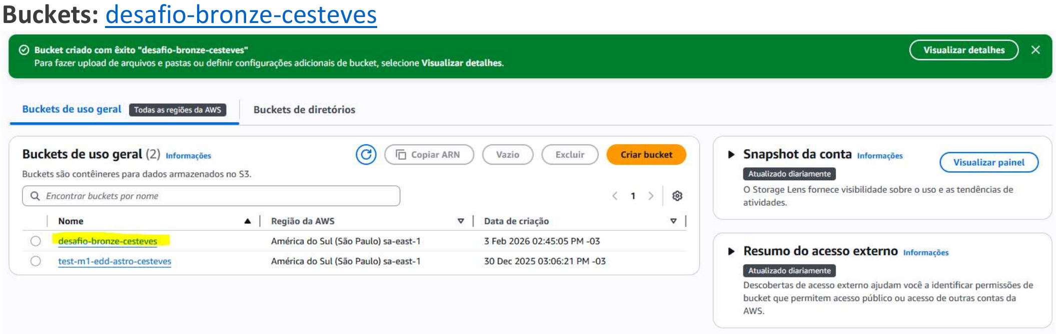


PASSO 3 — Criar o Bucket S3 Bronze (Data Lake):

Console AWS → Pesquise por S3 → "buckets" → "Criar bucket" → Dê nome ao bucket → Criar bucket.

Atenção: Tudo que conversa com o S3 precisa apontar para a MESMA região do bucket, neste caso: América do Sul (São Paulo) sa-east-1.

1. **Buckets:** desafio-bronze-cesteves

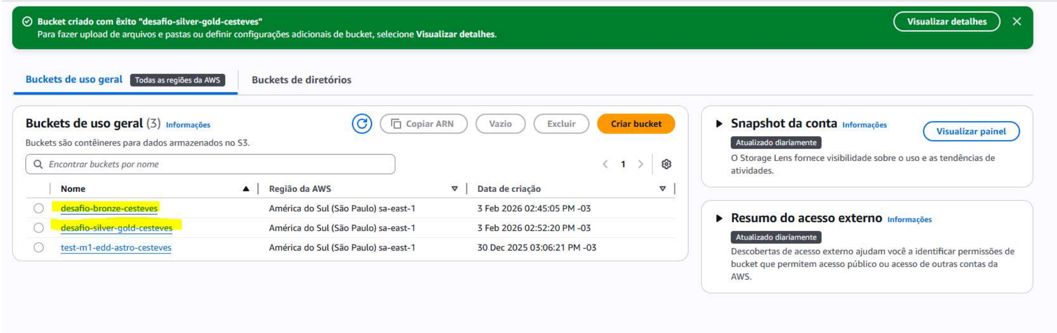


PASSO 4 — Criar o Bucket S3 Silver/Gold (Data Lake):

Console AWS → Pesquise por S3 → "buckets" → "Criar bucket" → Dê nome ao bucket → Criar bucket.

Atenção: Tudo que conversa com o S3 precisa apontar para a MESMA região do bucket, neste caso: América do Sul (São Paulo) sa-east-1.

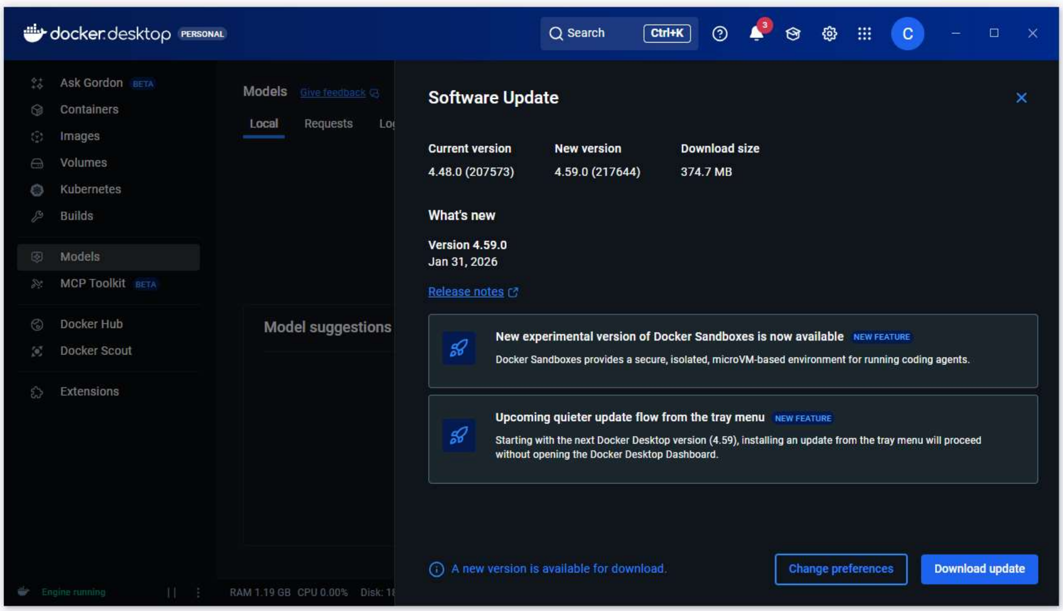
2. **Buckets:** [desafio-silver-gold-cesteves]()



PASSO 5 — Baixando e configurando o Docker:

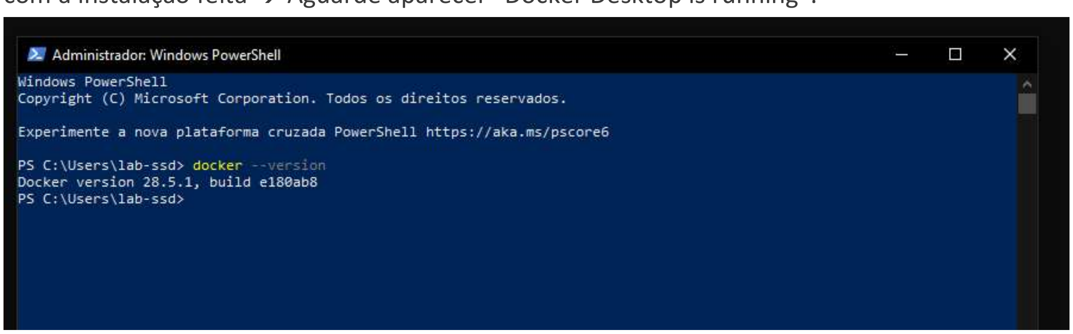
Acesse o site abaixo → "Baixar o Docker Desktop" → Execute → Marque a opção "Use WSL 2 instead of Hyper-V" (recomendado)" caso ela não esteja marcada → Reinicie a máquina.

Obs.: Já possui o Docker.



PASSO 6 — Acessando o Docker:

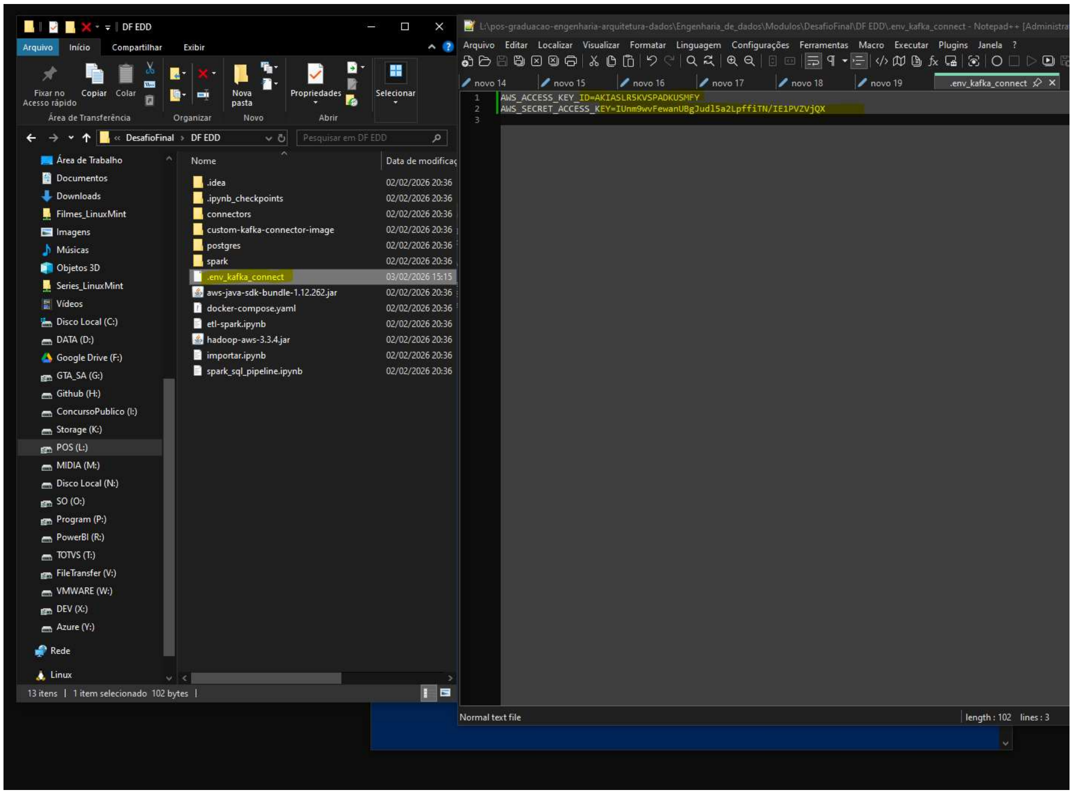
Abra o Docker Desktop → No poweshell execute o comando: docker --version para validar que está tudo certo com a instalação feita → Aguarde aparecer "Docker Desktop is running".



PASSO 7 — Editar o arquivo .env_kafka_connect:

Na raiz da pasta enviada pelo professor editado o arquivo de texto com a chave e a senha de acesso que anotamos no passo da AWS e renomeado com .env_kafka_connect.

L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD



PASSO 8 — Alterar os arquivos de conectores:

L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\connectors\sink → Altere ambos os arquivos

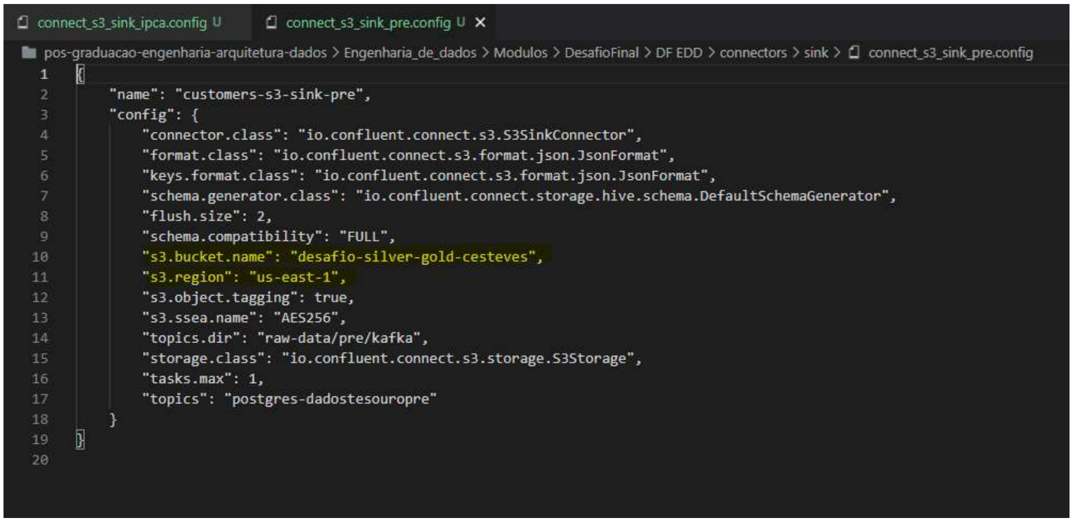
connect_s3_sink_ipca.config

connect_s3_sink_ipca.config U × connect_s3_sink_pre.config U

pos-graduacao-engenharia-arquitetura-dados > Engenharia_de_dados > Modulos > DesafioFinal > DF EDD > connectors > sink > connect_s3_sink_ipca.config

```
{
    "name": "customers-s3-sink-ipca",
    "config": {
        "connector.class": "io.confluent.connect.s3.S3SinkConnector",
        "format.class": "io.confluent.connect.s3.format.json.JsonFormat",
        "keys.format.class": "io.confluent.connect.s3.format.json.JsonFormat",
        "schema.generator.class": "io.confluent.connect.storage.hive.schema.DefaultSchemaGenerator",
        "flush.size": 2,
        "schema.compatibility": "FULL",
        "s3.bucket.name": "desafio-bronze-cesteves",
        "s3.region": "us-east-1",
        "s3.object.tagging": true,
        "s3.ssea.name": "AES256",
        "topics.dir": "raw-data/ipca/kafka",
        "storage.class": "io.confluent.connect.s3.storage.S3Storage",
        "tasks.max": 1,
        "topics": "postgres-dadostesouroipca"
    }
}
```

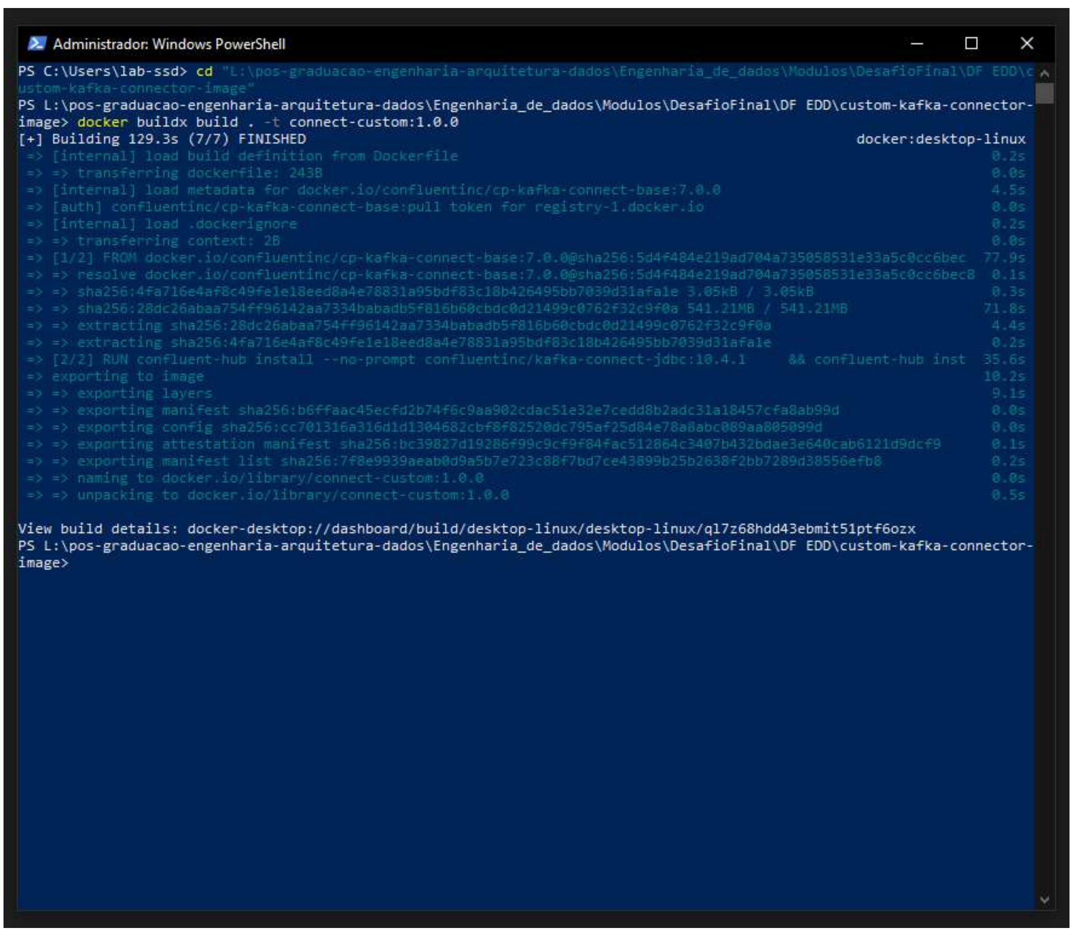
connect_s3_sink_pre.config



PASSO 9 — Build da imagem do Kafka Connect:

Abra o powershell e digite "cd + endereço onde está a imagem do kafka" → depois execute: docker buildx build . -t connect-custom:1.0.0

Neste desfio o caminho será: cd "L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\custom-kafka-connector-image"

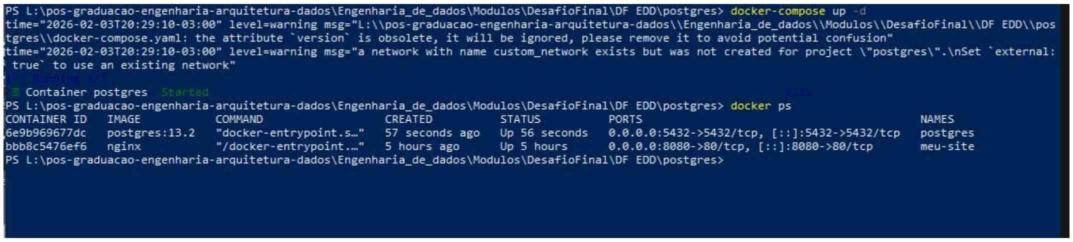


PASSO 10 — Subir o PostgreSQL:

Abra o powershell e digite "cd + endereço onde está PostgreSQL → depois execute: docker-compose up -d

Comando: cd “L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\postgres”

Comando: docker-compose up -d

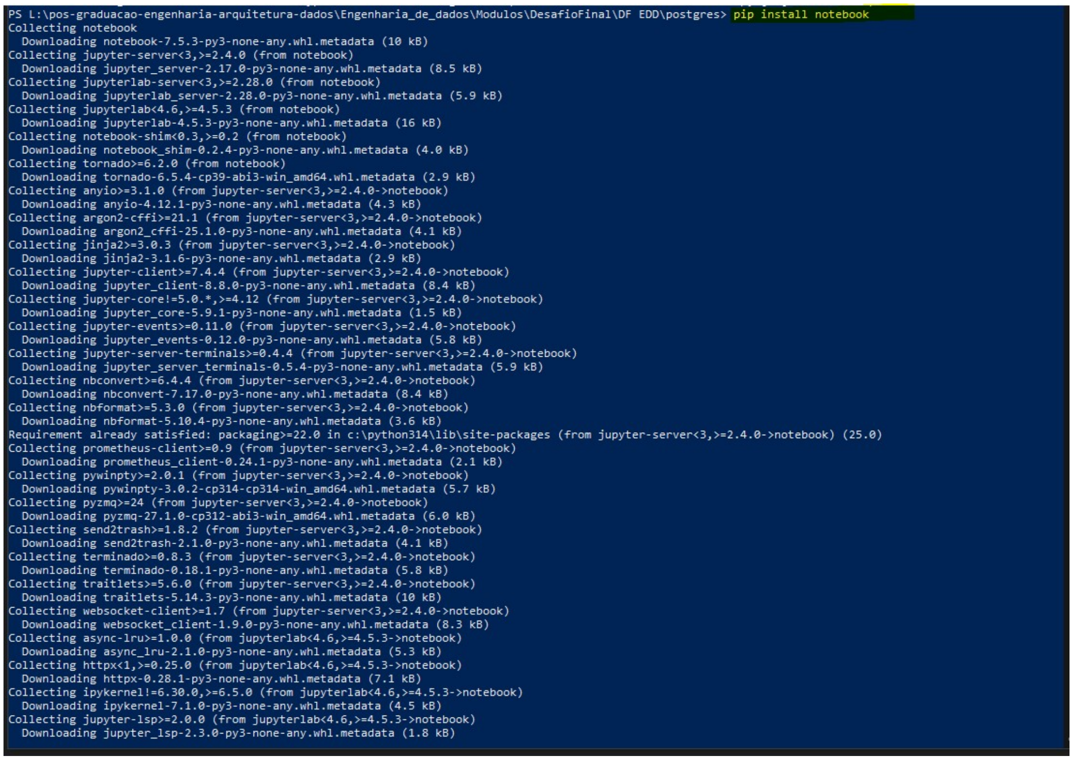


```
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\postgres> docker-compose up -d
time="2026-02-03T20:29:10-03:00" level=warning msg="L:\\pos-graduacao-engenharia-arquitetura-dados\\Engenharia_de_dados\\Modulos\\DesafioFinal\\DF EDD\\pos
tgres\\docker-compose.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
time="2026-02-03T20:29:10-03:00" level=warning msg="a network with name custom_network exists but was not created for project \"postgres\".\nSet `external:
 true` to use an existing network"
 ✔ Container postgres  Started
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\postgres> docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED          STATUS          PORTS                                         NAMES
6e9b969677dc   postgres:13.2   "docker-entrypoint.s…"   57 seconds ago   Up 56 seconds   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   postgres
bbb8c5476ef6   nginx           "/docker-entrypoint.…"   5 hours ago      Up 5 hours      0.0.0.0:8080->80/tcp, [::]:8080->80/tcp       meu-site
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\postgres>
```

PASSO 11 — Instalar Jupyter Notebook e importar dados:

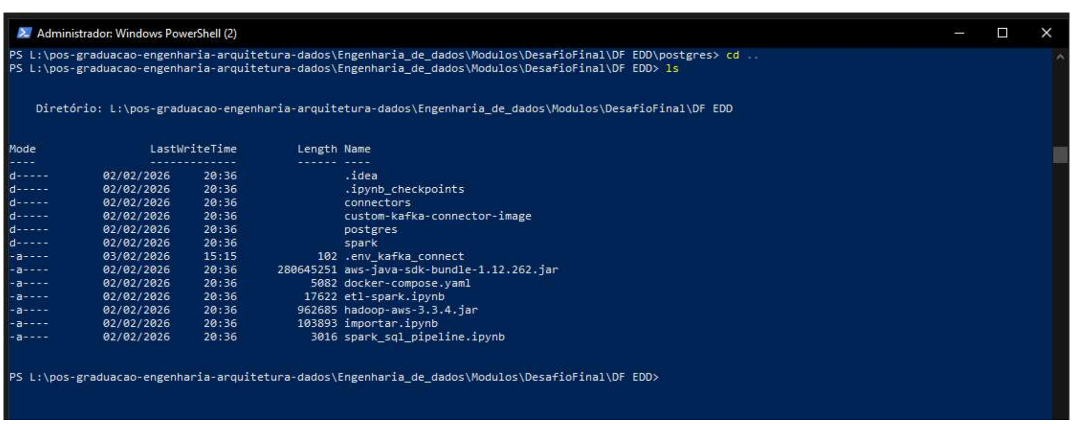
Abra o powershell e digite: pip install notebook → Ir para pasta do projeto → Abrir o Jupyter através do comando: jupyter notebook → Execute o notebook de importação

Instalação: pip install notebook



```
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\postgres> pip install notebook
Collecting notebook
  Downloading notebook-7.5.3-py3-none-any.whl.metadata (10 kB)
Collecting jupyter-server<3,>=2.4.0 (from notebook)
  Downloading jupyter_server-2.17.0-py3-none-any.whl.metadata (8.5 kB)
Collecting jupyterlab-server<3,>=2.28.0 (from notebook)
  Downloading jupyterlab_server-2.28.0-py3-none-any.whl.metadata (5.9 kB)
Collecting jupyterlab<4.6,>=4.5.3 (from notebook)
  Downloading jupyterlab-4.5.3-py3-none-any.whl.metadata (16 kB)
Collecting notebook-shim<0.3,>=0.2 (from notebook)
  Downloading notebook_shim-0.2.4-py3-none-any.whl.metadata (4.0 kB)
Collecting tornado>=6.2.0 (from notebook)
  Downloading tornado-6.5.4-cp39-abi3-win_amd64.whl.metadata (2.9 kB)
Collecting anyio>=3.1.0 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading anyio-4.12.1-py3-none-any.whl.metadata (4.3 kB)
Collecting argon2-cffi>=21.1 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading argon2_cffi-25.1.0-py3-none-any.whl.metadata (4.1 kB)
Collecting jinja2>=3.0.3 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting jupyter-client>=7.4.4 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading jupyter_client-8.8.0-py3-none-any.whl.metadata (8.4 kB)
Collecting jupyter-core!=5.0.*,>=4.12 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading jupyter_core-5.9.1-py3-none-any.whl.metadata (1.5 kB)
Collecting jupyter-events>=0.11.0 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading jupyter_events-0.12.0-py3-none-any.whl.metadata (5.8 kB)
Collecting jupyter-server-terminals>=0.4.4 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading jupyter_server_terminals-0.5.4-py3-none-any.whl.metadata (5.9 kB)
Collecting nbconvert>=6.4.4 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading nbconvert-7.17.0-py3-none-any.whl.metadata (8.4 kB)
Collecting nbformat>=5.3.0 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading nbformat-5.10.4-py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: packaging>=22.0 in c:\python314\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (25.0)
Collecting prometheus-client>=0.9 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading prometheus_client-0.24.1-py3-none-any.whl.metadata (2.1 kB)
Collecting pywinpty>=2.0.1 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading pywinpty-3.0.2-cp314-cp314-win_amd64.whl.metadata (5.7 kB)
Collecting pyzmq>=24 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading pyzmq-27.1.0-cp312-abi3-win_amd64.whl.metadata (6.0 kB)
Collecting send2trash>=1.8.2 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading send2trash-2.1.0-py3-none-any.whl.metadata (4.1 kB)
Collecting terminado>=0.8.3 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading terminado-0.18.1-py3-none-any.whl.metadata (5.8 kB)
Collecting traitlets>=5.6.0 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading traitlets-5.14.3-py3-none-any.whl.metadata (10 kB)
Collecting websocket-client>=1.7 (from jupyter-server<3,>=2.4.0->notebook)
  Downloading websocket_client-1.9.0-py3-none-any.whl.metadata (8.3 kB)
Collecting async-lru>=1.0.0 (from jupyterlab<4.6,>=4.5.3->notebook)
  Downloading async_lru-2.1.0-py3-none-any.whl.metadata (5.3 kB)
Collecting httpx<1,>=0.25.0 (from jupyterlab<4.6,>=4.5.3->notebook)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting ipykernel!=6.30.0,>=6.5.0 (from jupyterlab<4.6,>=4.5.3->notebook)
  Downloading ipykernel-7.1.0-py3-none-any.whl.metadata (4.5 kB)
Collecting jupyter-lsp>=2.0.0 (from jupyterlab<4.6,>=4.5.3->notebook)
  Downloading jupyter_lsp-2.3.0-py3-none-any.whl.metadata (1.8 kB)
```

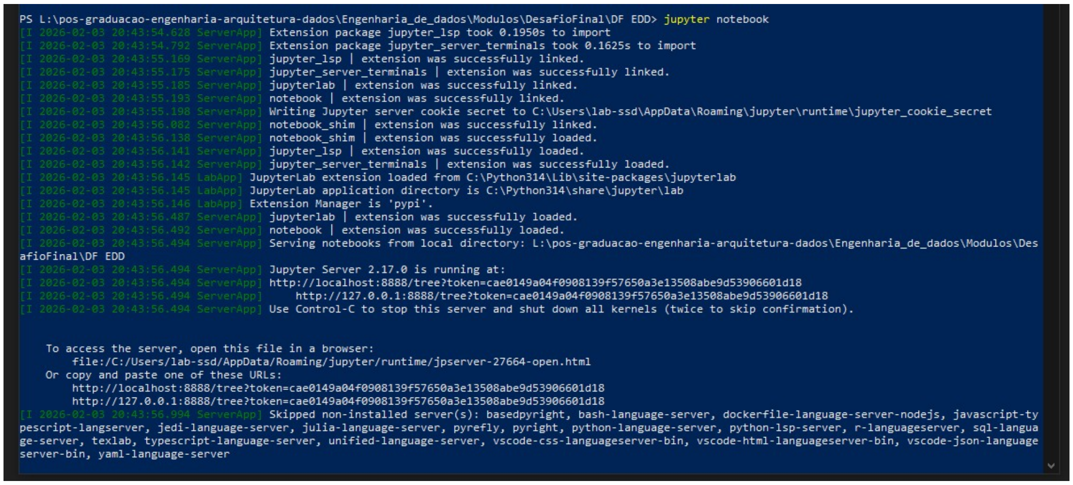
Pasta do projeto:



```
Administrador: Windows PowerShell (2)
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\postgres> cd ..
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD> ls


    Diretório: L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        02/02/2026     20:36                .idea
d-----        02/02/2026     20:36                .ipynb_checkpoints
d-----        02/02/2026     20:36                connectors
d-----        02/02/2026     20:36                custom-kafka-connector-image
d-----        02/02/2026     20:36                postgres
d-----        02/02/2026     20:36                spark
-a----        03/02/2026     15:15            102 .env_kafka_connect
-a----        02/02/2026     20:36      280645251 aws-java-sdk-bundle-1.12.262.jar
-a----        02/02/2026     20:36           5082 docker-compose.yaml
-a----        02/02/2026     20:36          17622 etl-spark.ipynb
-a----        02/02/2026     20:36         962685 hadoop-aws-3.3.4.jar
-a----        02/02/2026     20:36         103893 importar.ipynb
-a----        02/02/2026     20:36           3016 spark_sql_pipeline.ipynb


PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD>
```

Abrindo o jupyter notebook dentro do projeto:

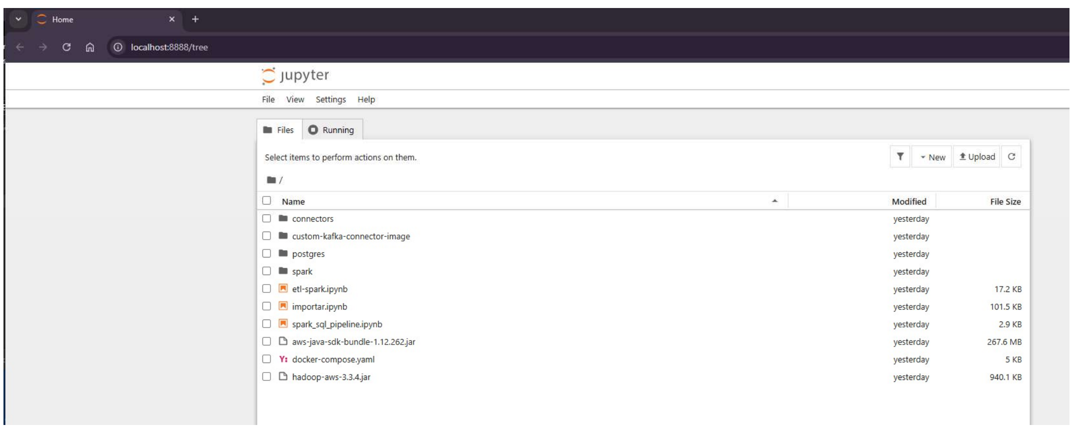
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD> jupyter notebook



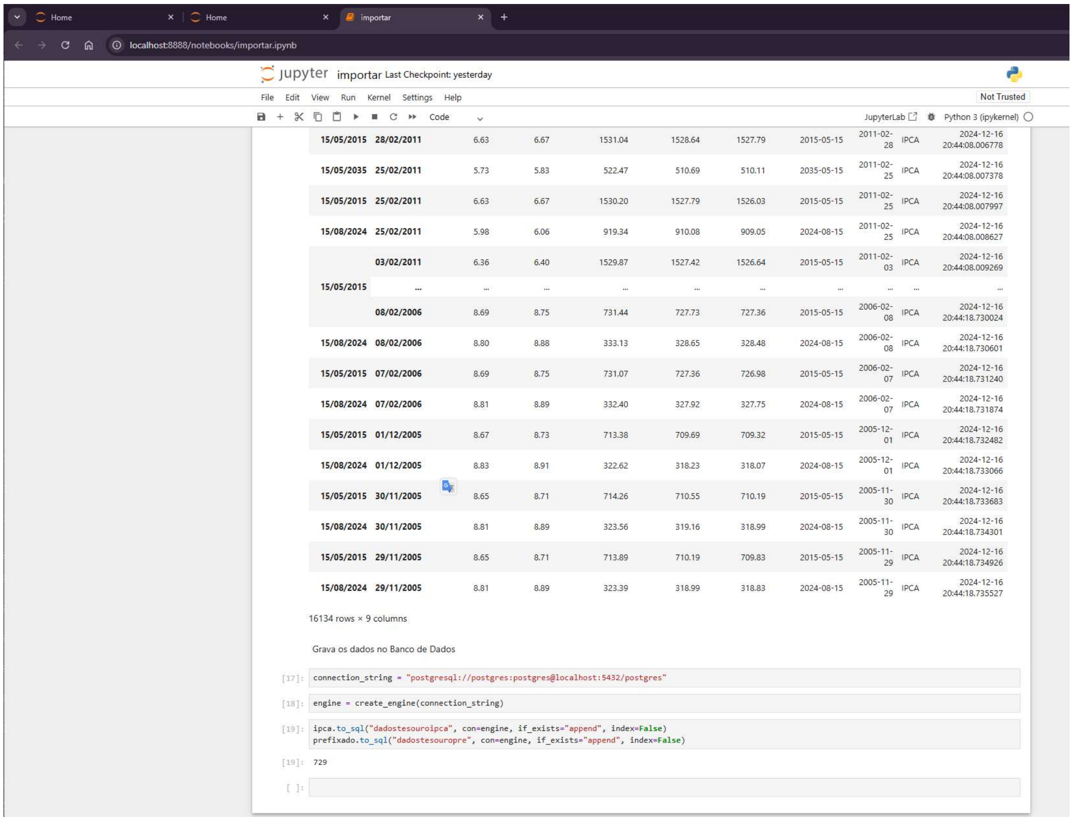
Homepage do jupyter

http://localhost:8888/tree

http://localhost:8888/tree?token=6c55d27429f3bb2a04657dbbe51795a92c83fb596e1b7aa2



importar.ipynb



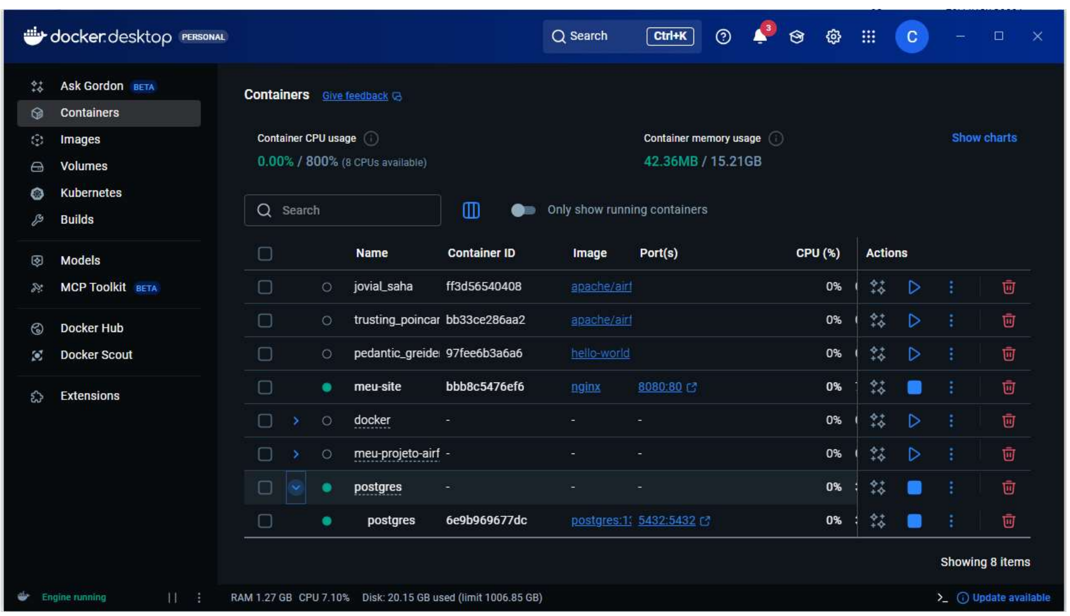
15/05/2015	28/02/2011	6.63	6.67	1531.04	1528.64	1527.79	2015-05-15	2011-02-28	IPCA	2024-12-16 20:44:08.006778
15/05/2035	25/02/2011	5.73	5.83	522.47	510.69	510.11	2035-05-15	2011-02-25	IPCA	2024-12-16 20:44:08.007378
15/05/2015	25/02/2011	6.63	6.67	1530.20	1527.79	1526.03	2015-05-15	2011-02-25	IPCA	2024-12-16 20:44:08.007997
15/08/2024	25/02/2011	5.98	6.06	919.34	910.08	909.05	2024-08-15	2011-02-25	IPCA	2024-12-16 20:44:08.008627
	03/02/2011	6.36	6.40	1529.87	1527.42	1526.64	2015-05-15	2011-02-03	IPCA	2024-12-16 20:44:08.009269
15/05/2015	...	...	...	...	...	...	...	...	...	...
	08/02/2006	8.69	8.75	731.44	727.73	727.36	2015-05-15	2006-02-08	IPCA	2024-12-16 20:44:18.730024
15/08/2024	08/02/2006	8.80	8.88	333.13	328.65	328.48	2024-08-15	2006-02-08	IPCA	2024-12-16 20:44:18.730601
15/05/2015	07/02/2006	8.69	8.75	731.07	727.36	726.98	2015-05-15	2006-02-07	IPCA	2024-12-16 20:44:18.731240
15/08/2024	07/02/2006	8.81	8.89	332.40	327.92	327.75	2024-08-15	2006-02-07	IPCA	2024-12-16 20:44:18.731874
15/05/2015	01/12/2005	8.67	8.73	713.38	709.69	709.32	2015-05-15	2005-12-01	IPCA	2024-12-16 20:44:18.732482
15/08/2024	01/12/2005	8.83	8.91	322.62	318.23	318.07	2024-08-15	2005-12-01	IPCA	2024-12-16 20:44:18.733066
15/05/2015	30/11/2005	8.65	8.71	714.26	710.55	710.19	2015-05-15	2005-11-30	IPCA	2024-12-16 20:44:18.733683
15/08/2024	30/11/2005	8.81	8.89	323.56	319.16	318.99	2024-08-15	2005-11-30	IPCA	2024-12-16 20:44:18.734301
15/05/2015	29/11/2005	8.65	8.71	713.89	710.19	709.83	2015-05-15	2005-11-29	IPCA	2024-12-16 20:44:18.734926
15/08/2024	29/11/2005	8.81	8.89	323.39	318.99	318.83	2024-08-15	2005-11-29	IPCA	2024-12-16 20:44:18.735527

16134 rows × 9 columns

Grava os dados no Banco de Dados

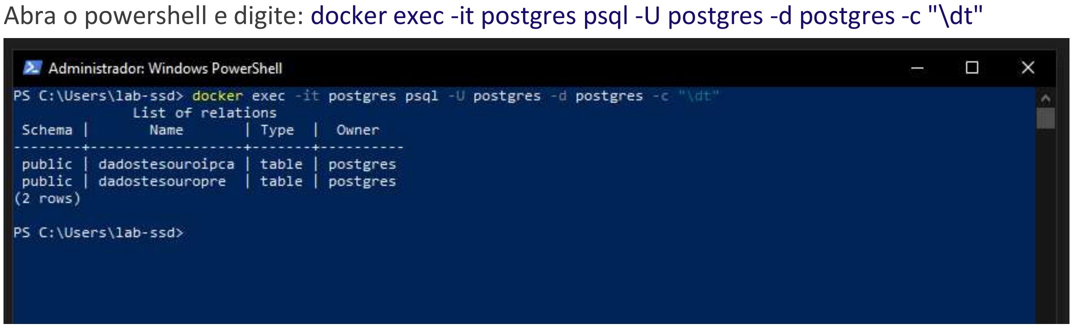
```
[17]: connection_string = "postgresql://postgres:postgres@localhost:5432/postgres"
[18]: engine = create_engine(connection_string)
[19]: ipca.to_sql("dadostesouroipca", con=engine, if_exists="append", index=False)
      prefixado.to_sql("dadostesouropre", con=engine, if_exists="append", index=False)
[19]: 729
[ ]:
```

postgres rodando no docker



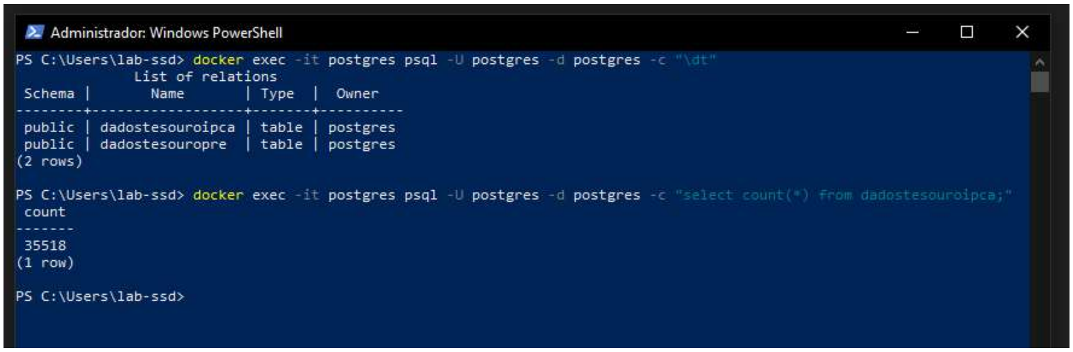
PASSO 12 — Validar se os dados foram gravados no postgres:

Abra o powershell e digite: docker exec -it postgres psql -U postgres -d postgres -c "\dt"



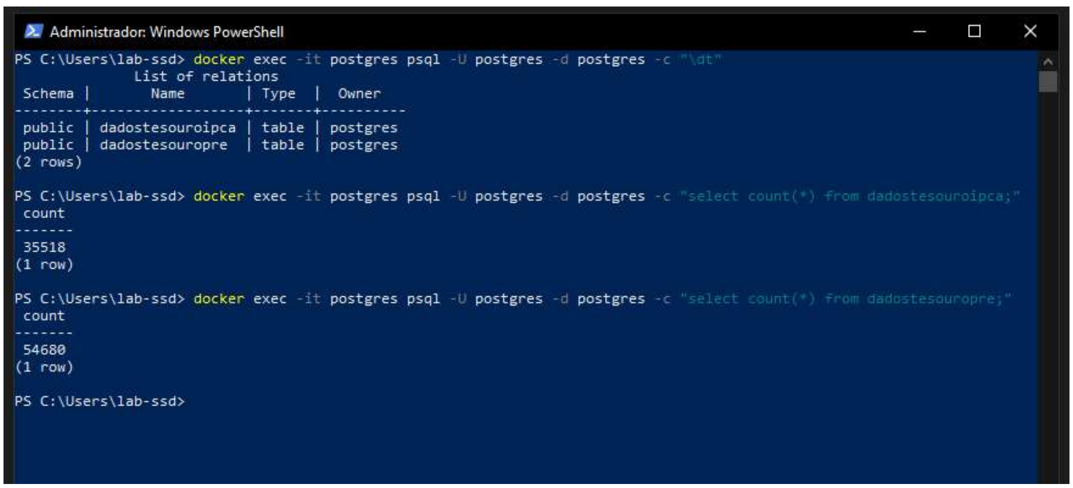
Conte as linhas: docker exec -it postgres psql -U postgres -d postgres -c "select count(*) from dadostesouroipca;"

dadostesouroipca



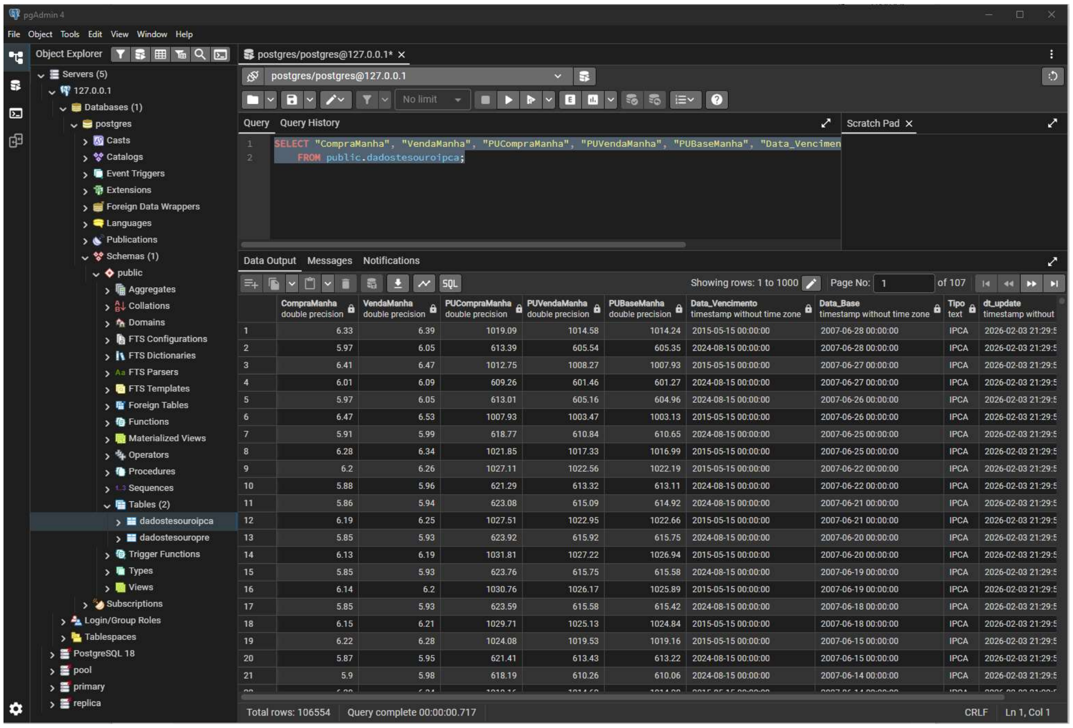
Conte as linhas: docker exec -it postgres psql -U postgres -d postgres -c "select count(*) from dadostesouropre;"

Dadostesouropre

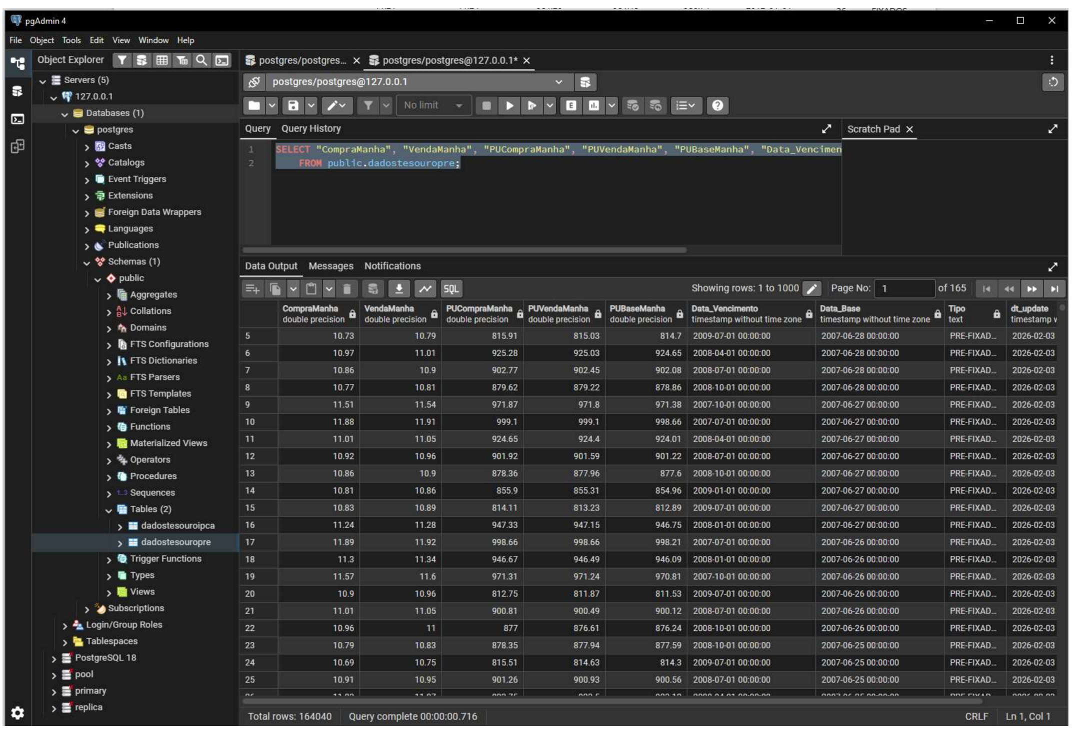


Postgres:

Dadostesouroipca



Dadostesouropre



PASSO 13 — Subir a Plataforma Kafka + Kafka Connect:

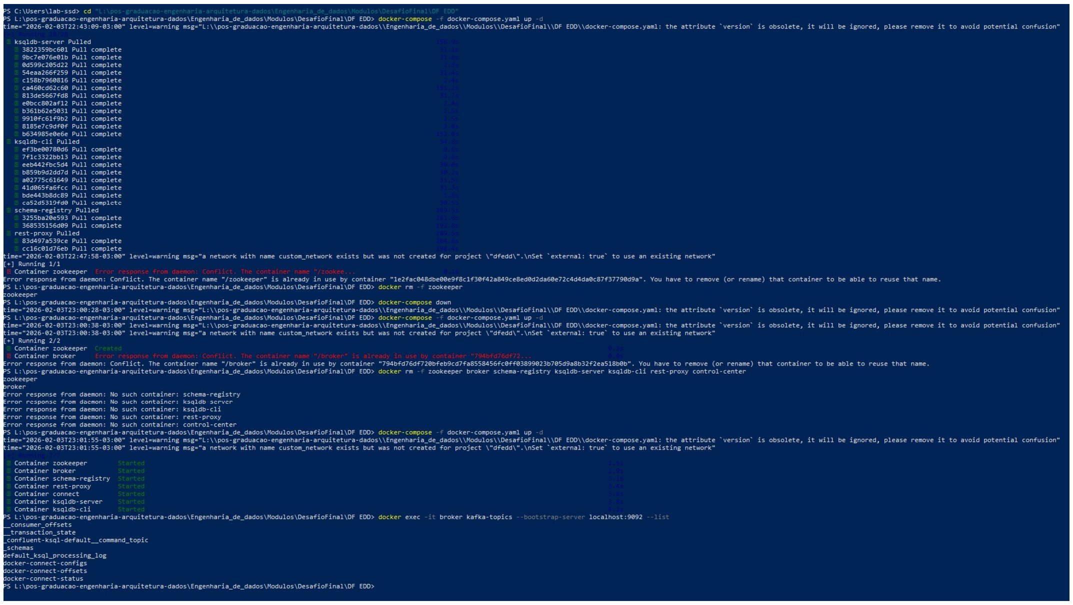
Abra o powershell e digite:

cd + Raiz do projeto → Suba o ecossistema Kafka através do comando: docker-compose -f docker-compose.yaml up -d

Obs.: Tive um problema com o zookeeper, uma "zumbi" bloqueando o caminho, foi resolvido com o comando abaixo:

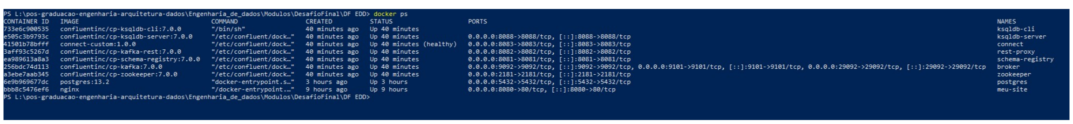
1. Remover o container conflitante (Zookeeper): docker rm -f zookeeper

2. Limpar possíveis conflitos dos outros serviços: docker-compose down
3. Subir o ambiente novamente: O docker-compose -f docker-compose.yaml up -d



Valide através do comando: docker ps

Neste caso será: “L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD”



PASSO 14 — Criar tópicos no Kafka:

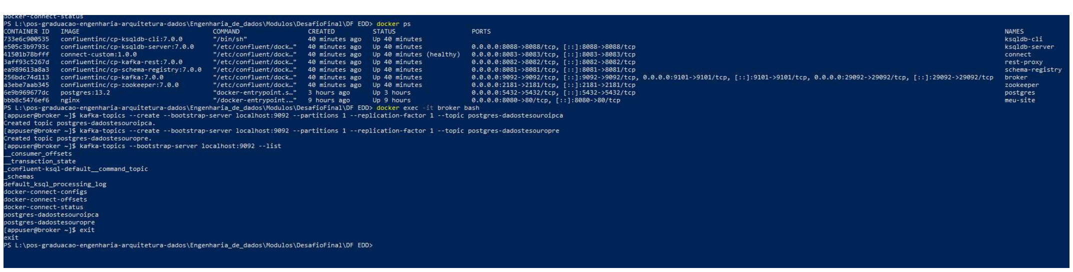
Abra o powershell e digite:

docker exec -it broker bash e depois digite: kafka-topics --create --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1 --topic postgres-dadostesouroipca

kafka-topics --create --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1 --topic postgres-dadostesouropre

kafka-topics --bootstrap-server localhost:9092 --list

exit



PASSO 15 — Registrar Source Connectors (Postgres → Kafka):

Abra o powershell e digite os seguintes comandos:

Para o 550 connect_jdbc_postgres_ipca.config:

curl.exe -X POST -H "Content-Type: application/json" --data-binary "@connect_jdbc_postgres_ipca.config" http://localhost:8083/connectors

Para o 544 connect_jdbc_postgres_pre.config

curl.exe -X POST -H "Content-Type: application/json" --data-binary "@connect_jdbc_postgres_pre.config" http://localhost:8083/connectors

PASSO 16 — Validar registro:

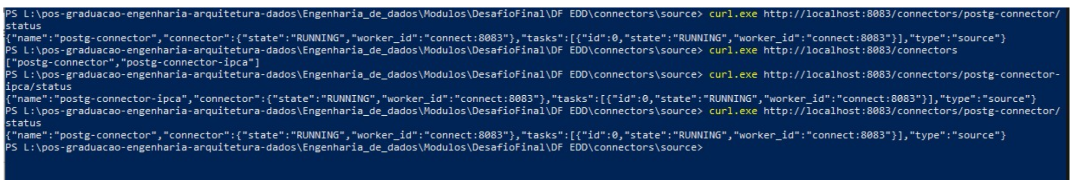
Abra o powershell e digite:

curl.exe http://localhost:8083/connectors

depois digite:

curl.exe http://localhost:8083/connectors/postg-connector-ipca/status

curl.exe http://localhost:8083/connectors/postg-connector/status



PASSO 17 — Confirmar que chegou dado nos tópicos (Postgres → Kafka):

Abra o powershell e digite:

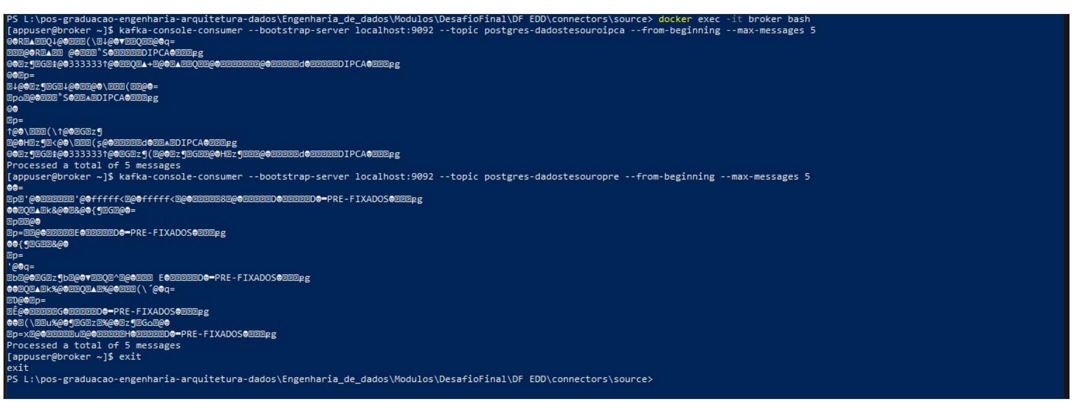
docker exec -it broker bash

Depois execute:

kafka-console-consumer --bootstrap-server localhost:9092 --topic postgres-dadostesouroipca --from-beginning --max-messages 5

kafka-console-consumer --bootstrap-server localhost:9092 --topic postgres-dadostesouropre --from-beginning --max-messages 5

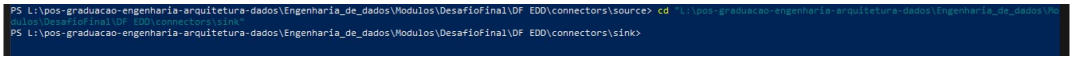
exit



PASSO 18 — Configurar e registrar os Sink Connectors (Kafka → S3):

Abra o powershell e digite:

cd “L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DF EDD\connectors\sink”

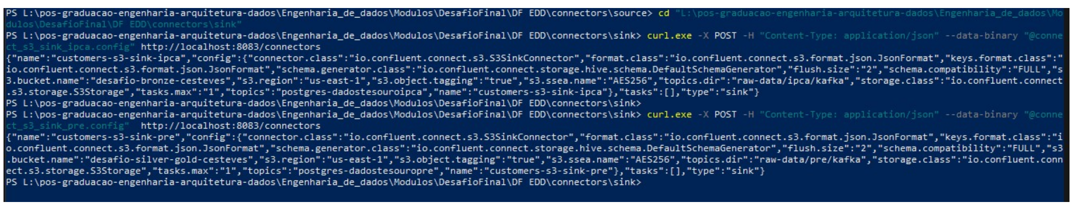


PASSO 19 — Registre (mesmo ajuste do curl):

Abra o powershell e digite:

curl.exe -X POST -H "Content-Type: application/json" --data-binary "@connect_s3_sink_ipca.config" http://localhost:8083/connectors

curl.exe -X POST -H "Content-Type: application/json" --data-binary "@connect_s3_sink_pre.config" http://localhost:8083/connectors



PASSO 20 — Validar status:

Abra o powershell e digite:

curl.exe http://localhost:8083/connectors/customers-s3-sink-ipca/status

curl.exe http://localhost:8083/connectors/customers-s3-sink-pre/status

PASSO 21 — Realizar ajustes nas configurações do etl-spark.ipynb

Caminho para a camada bronze no S3

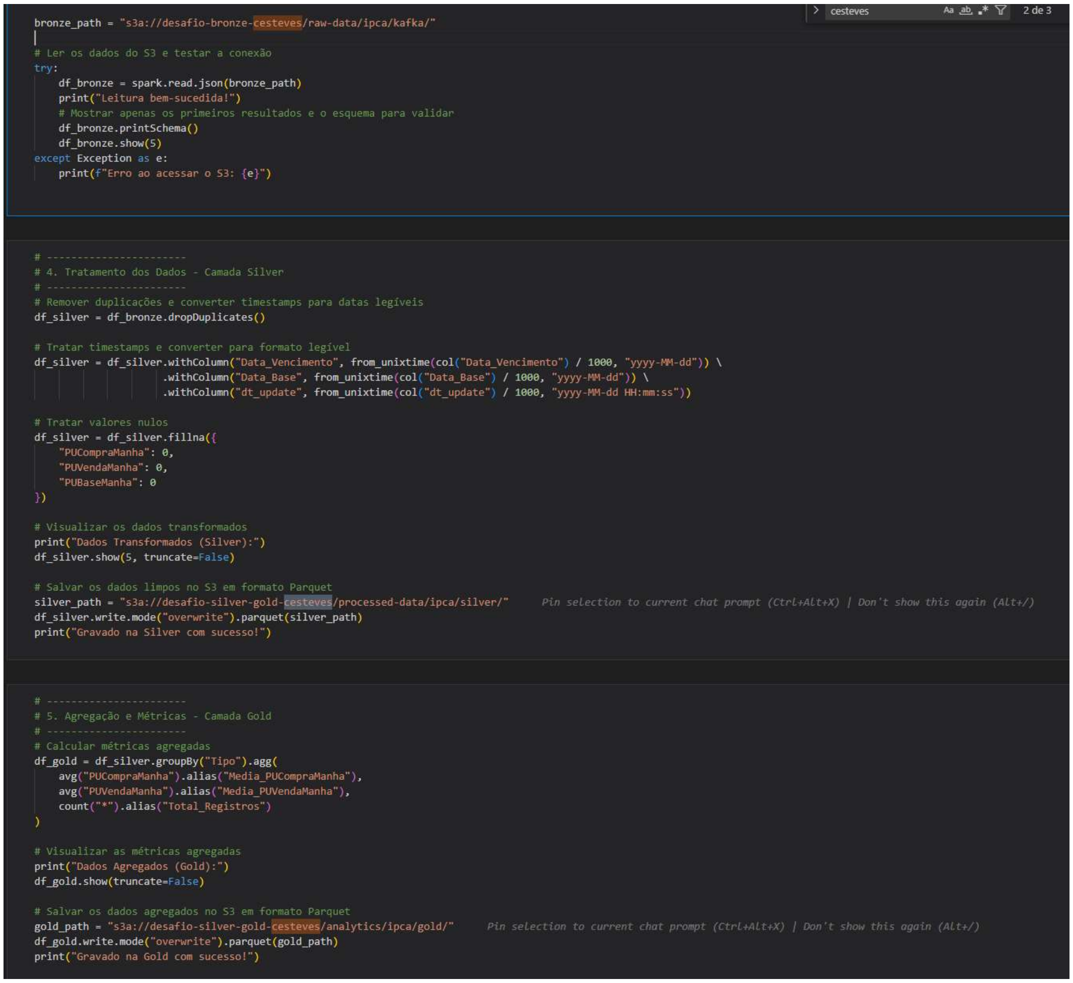
bronze_path = "s3a://desafio-bronze-cesteves/raw-data/ipca/kafka/"

Salvar os dados limpos no S3 em formato Parquet

silver_path = "s3a://desafio-silver-gold-cesteves/processed-data/ipca/silver/"

Salvar os dados agregados no S3 em formato Parquet

gold_path = "s3a://desafio-silver-gold-cesteves/analytics/ipca/gold/"



```
bronze_path = "s3a://desafio-bronze-cesteves/raw-data/ipca/kafka/"

# Ler os dados do S3 e testar a conexão
try:
    df_bronze = spark.read.json(bronze_path)
    print("Leitura bem-sucedida!")
    # Mostrar apenas os primeiros resultados e o esquema para validar
    df_bronze.printSchema()
    df_bronze.show(5)
except Exception as e:
    print(f"Erro ao acessar o S3: {e}")
```

```
# ----------------------
# 4. Tratamento dos Dados - Camada Silver
# ----------------------
# Remover duplicações e converter timestamps para datas legíveis
df_silver = df_bronze.dropDuplicates()

# Tratar timestamps e converter para formato legível
df_silver = df_silver.withColumn("Data_Vencimento", from_unixtime(col("Data_Vencimento") / 1000, "yyyy-MM-dd")) \
                     .withColumn("Data_Base", from_unixtime(col("Data_Base") / 1000, "yyyy-MM-dd")) \
                     .withColumn("dt_update", from_unixtime(col("dt_update") / 1000, "yyyy-MM-dd HH:mm:ss"))

# Tratar valores nulos
df_silver = df_silver.fillna({
    "PUCompraManha": 0,
    "PUVendaManha": 0,
    "PUBaseManha": 0
})

# Visualizar os dados transformados
print("Dados Transformados (Silver):")
df_silver.show(5, truncate=False)

# Salvar os dados limpos no S3 em formato Parquet
silver_path = "s3a://desafio-silver-gold-cesteves/processed-data/ipca/silver/"     Pin selection to current chat prompt (Ctrl+Alt+X) | Don't show this again (Alt+/)
df_silver.write.mode("overwrite").parquet(silver_path)
print("Gravado na Silver com sucesso!")
```

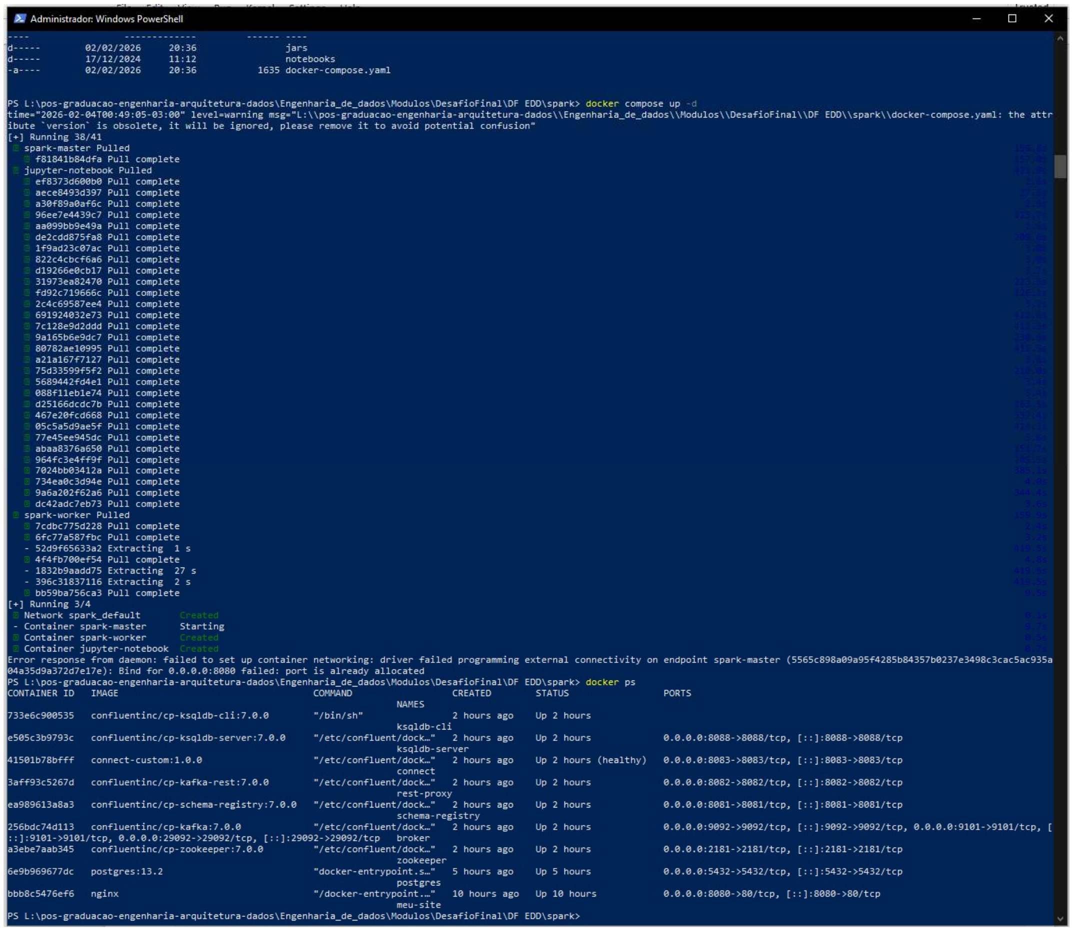
```
# ----------------------
# 5. Agregação e Métricas - Camada Gold
# ----------------------
# Calcular métricas agregadas
df_gold = df_silver.groupBy("Tipo").agg(
    avg("PUCompraManha").alias("Media_PUCompraManha"),
    avg("PUVendaManha").alias("Media_PUVendaManha"),
    count("*").alias("Total_Registros")
)

# Visualizar as métricas agregadas
print("Dados Agregados (Gold):")
df_gold.show(truncate=False)

# Salvar os dados agregados no S3 em formato Parquet
gold_path = "s3a://desafio-silver-gold-cesteves/analytics/ipca/gold/"     Pin selection to current chat prompt (Ctrl+Alt+X) | Don't show this again (Alt+/)
df_gold.write.mode("overwrite").parquet(gold_path)
print("Gravado na Gold com sucesso!")
```

PASSO 22 — Subir o Spark:

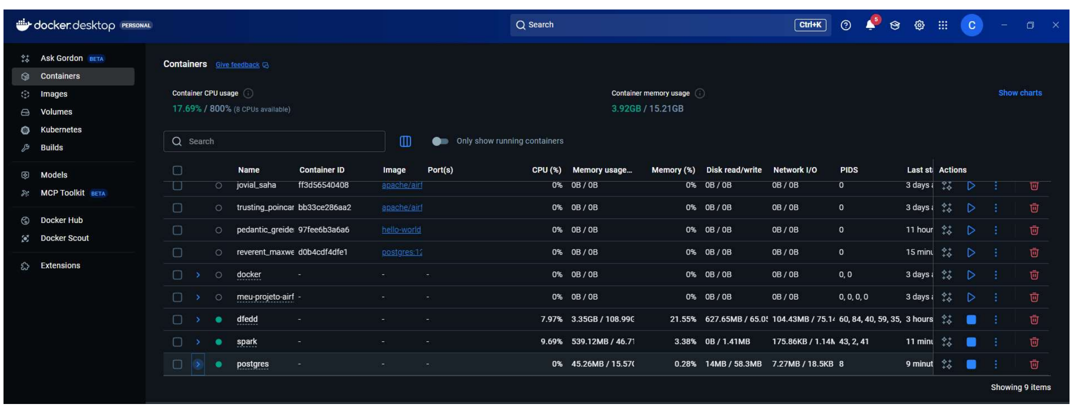
PS L:\pos-graduacao-engenharia-arquitetura-dados\Engenharia_de_dados\Modulos\DesafioFinal\DFEDD\spark>

Comando:

docker compose up -d



Docker



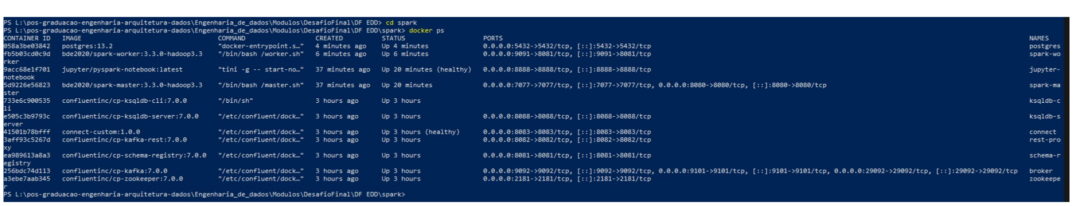
Spark Masterhttp://localhost:8080 Ver os Workers estão vivos.

Spark Workerhttp://localhost:9091 (Alterado da 8081 para 9091 para fugir do conflito).

Jupyter Notebookhttp://localhost:8888 Onde ver codar.

PASSO 23 — Confirmar que o container do Jupyter do Spark subiu:

Abra o powershell e digite: docker ps

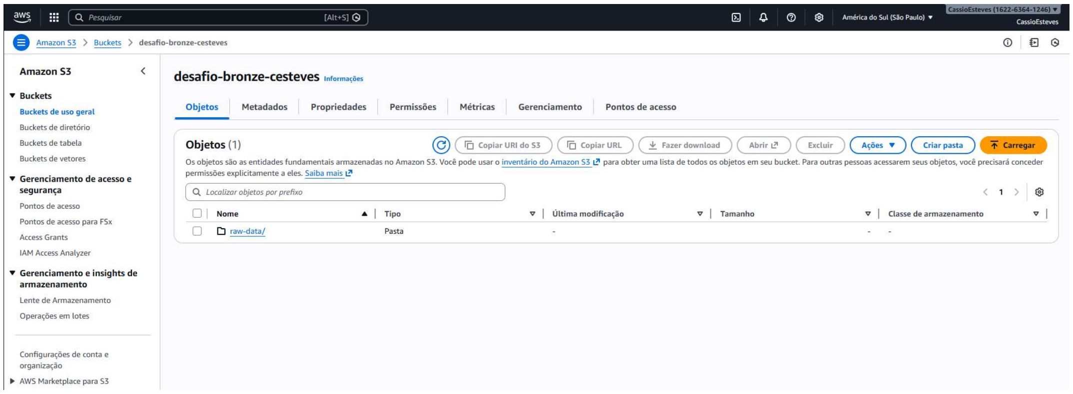


PASSO 24 — Confirmar que gravou no S3 (Entregável obrigatório):

1. Bronze

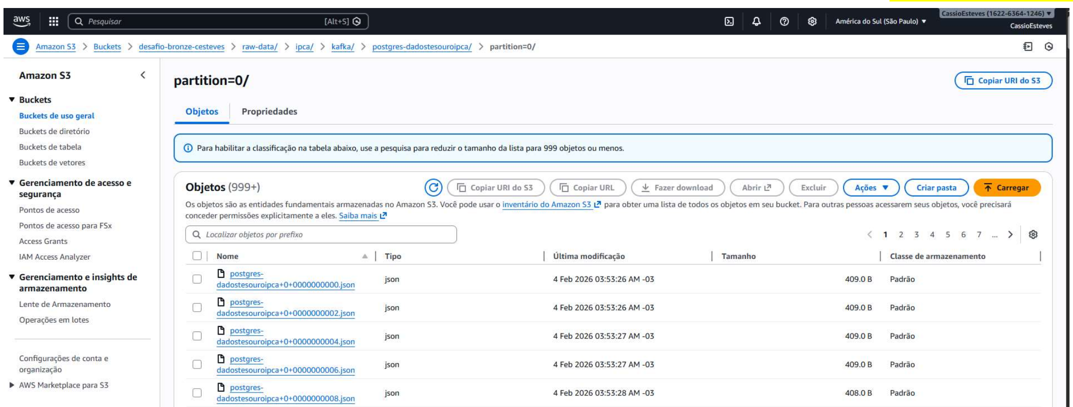
Abra desafio-bronze-cesteves

confirme a existência dos prefixos configurados

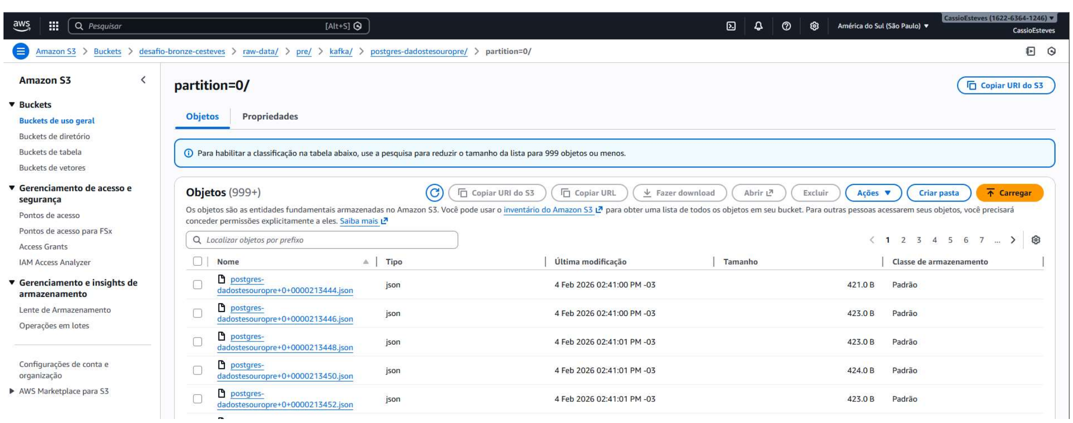


Confirmação de escrita:

Amazon S3 Buckets desafio-bronze-cesteves raw-data/ ipca/ kafka/ postgres-dadostesouroipca/ partition=0/



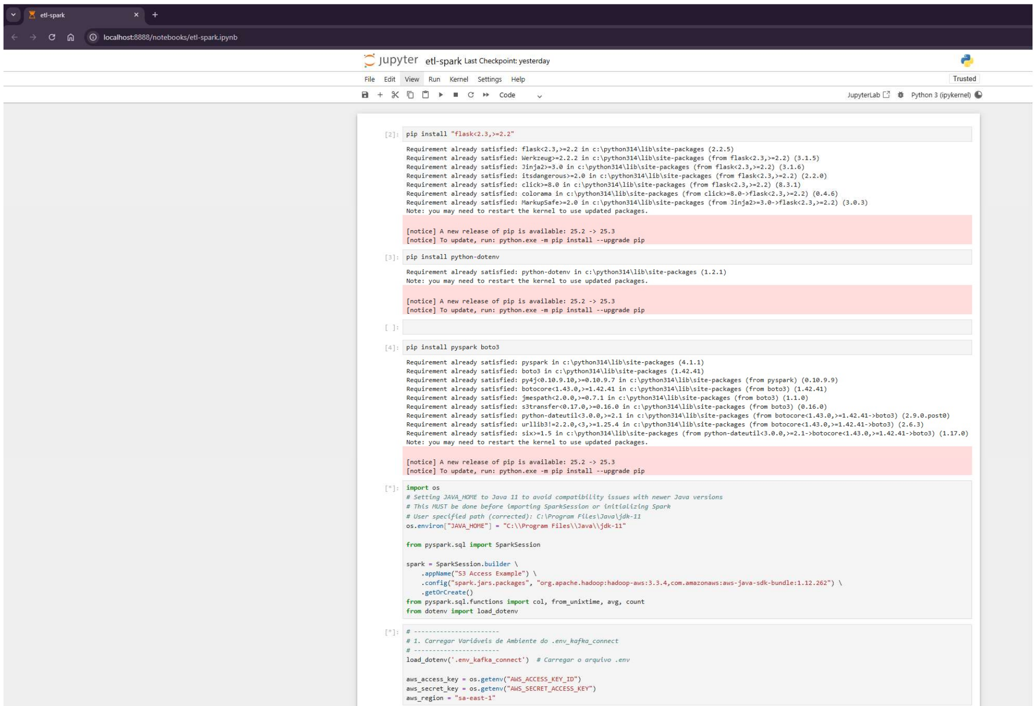
Amazon S3 Buckets desafio-bronze-cesteves raw-data/ pre/ kafka/ postgres-dadostesouropre/ partition=0/



2. Silver

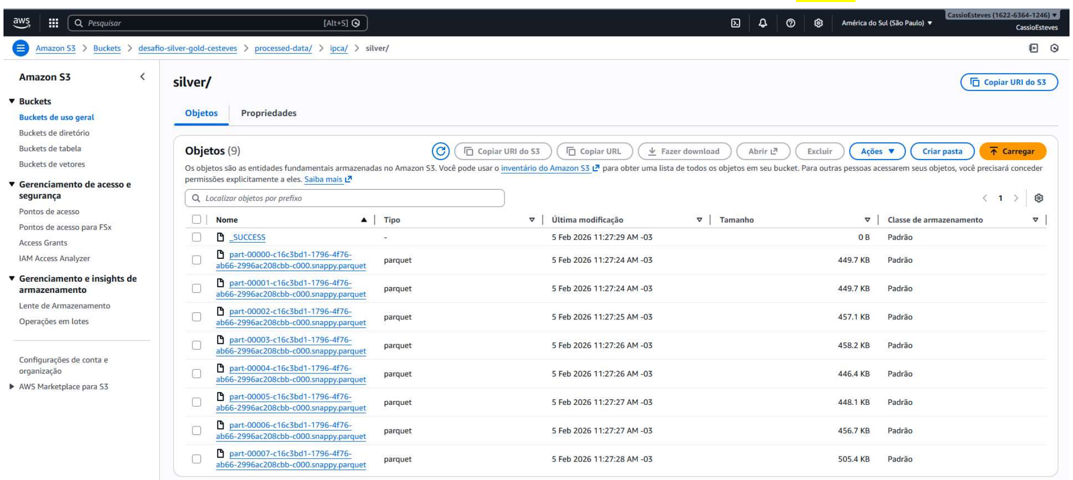
Camada SILVER — Limpeza e Padronização (Spark):

Abrir o Notebook etl-spark.ipynb → Execute



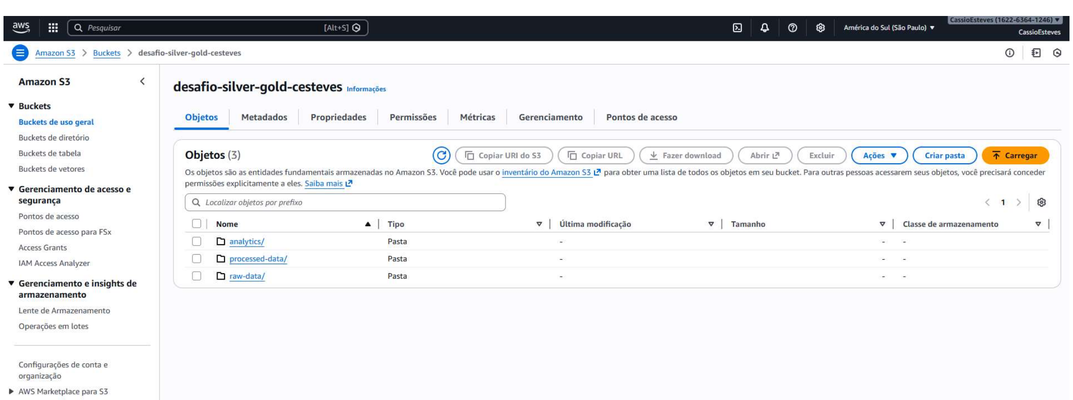
Confirmação de escrita:

Amazon S3 Buckets desafio-silver-gold-cesteves processed-data/ ipca/ silver/



confirme a existência dos prefixos configurados

desafio-silver-gold-cesteves



3. Gold

Amazon S3 Buckets desafio-silver-gold-cesteves analytics/ ipca/ gold/